

Competitive Programming

Section 72 Editorial: Master+ Flow And Convex Optimization Mixed Contest

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Exam Room Bounds	4
2.1	How to find it	4
2.2	Correctness	4
2.3	Complexity and tests	4
2.4	C++	4
2.5	Python	6
3	B. Bounded Exact Shipment	9
4	C. Selective Quota Profit	14
4.1	How to find it	14
4.2	Correctness	14
4.3	Complexity and tests	14
4.4	C++	14
4.5	Python	17
5	D. Bounded Convex Allocation	20
5.1	How to find it	20
5.2	Correctness	20
5.3	Complexity and tests	20
5.4	C++	20
5.5	Python	23
6	E. Exact Red Forest	25
6.1	How to find it	25
6.2	Correctness	25
6.3	Complexity and tests	25
6.4	C++	25
6.5	Python	27
7	F. Congested Team Assignment	30
7.1	How to find it	30
7.2	Correctness	30
7.3	Complexity and tests	30
7.4	C++	30
7.5	Python	33

1 How to use this editorial

For each problem, identify the reduction before reading the code. The proof paragraph names the invariant that should have appeared in your own derivation.

2 A. Exam Room Bounds

Create source-to-group edges fixed at each demand, allowed unit group-to-room edges, room-to-sink bounded edges, and a sink-to-source edge fixed at total demand. This is a bounded circulation, so the balance reduction decides feasibility.

2.1 How to find it

Rows and columns are two sides of a bipartite unit-flow model. Row demands are fixed source-edge flows, room limits are lower/upper sink-edge bounds, and an allowed pair is a unit middle edge. Fixing the return edge to the total demand turns the whole construction into a circulation.

2.2 Correctness

A valid selection sends one unit through exactly each selected pair; row and column constraints become the stated edge bounds, so it produces a feasible circulation. Conversely, integral max flow makes every middle edge zero or one. Reading its unit edges gives distinct allowed choices, with exact row degrees and bounded column degrees. The bounded-circulation equivalence then decides the instance exactly.

2.3 Complexity and tests

The graph has $R+C+4$ vertices and $E+R+C+O(1)$ edges. Test duplicate-looking constraints, a room minimum unreachable from enough groups, zero row demands, and feasible totals whose allowed-pair structure is nevertheless impossible.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;

struct Dinic {
    struct Edge {
        int to, rev;
        int64 cap;
    };
    vector<vector<Edge>> g;
```

```

vector<int> level, it;
Dinic(int n) : g(n), level(n), it(n) {}
void add_edge(int u, int v, int64 cap) {
    Edge a{v, (int)g[v].size(), cap};
    Edge b{u, (int)g[u].size(), 0};
    g[u].push_back(a);
    g[v].push_back(b);
}
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    level[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto &e : g[u])
            if (e.cap && level[e.to] < 0)
                level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] >= 0;
}
int64 dfs(int u, int t, int64 pushed) {
    if (u == t)
        return pushed;
    for (int &i = it[u]; i < (int)g[u].size(); ++i) {
        Edge &e = g[u][i];
        if (!e.cap || level[e.to] != level[u] + 1)
            continue;
        int64 take = dfs(e.to, t, min(pushed, e.cap));
        if (take) {
            e.cap -= take;
            g[e.to][e.rev].cap += take;
            return take;
        }
    }
    return 0;
}
int64 flow(int s, int t) {
    int64 ans = 0, pushed;
    while (bfs(s, t)) {
        fill(it.begin(), it.end(), 0);
        while ((pushed = dfs(s, t, (1LL << 62))))
            ans += pushed;
    }
    return ans;
}
};

int main() {
    ios::sync_with_stdio(false);

```

```

cin.tie(nullptr);
int rows, cols, m;
if (!(cin >> rows >> cols >> m))
    return 0;
int n = rows + cols + 2;
int source = rows + cols, sink = source + 1, ss = n, tt = n + 1;
Dinic dinic(n + 2);
vector<int64> balance(n);
int64 total = 0;
auto bounded = [&](int u, int v, int64 low, int64 high) {
    dinic.add_edge(u, v, high - low);
    balance[u] -= low;
    balance[v] += low;
};
for (int i = 0; i < rows; ++i) {
    int64 demand;
    cin >> demand;
    bounded(source, i, demand, demand);
    total += demand;
}
for (int j = 0; j < cols; ++j) {
    int64 low, high;
    cin >> low >> high;
    bounded(rows + j, sink, low, high);
}
for (int i = 0, r, c; i < m; ++i) {
    cin >> r >> c;
    bounded(--r, rows + --c, 0, 1);
}
bounded(sink, source, total, total);
int64 need = 0;
for (int v = 0; v < n; ++v) {
    if (balance[v] > 0)
        dinic.add_edge(ss, v, balance[v]), need += balance[v];
    if (balance[v] < 0)
        dinic.add_edge(v, tt, -balance[v]);
}
cout << (dinic.flow(ss, tt) == need ? "YES\n" : "NO\n");
}

```

2.5 Python

```

import collections
import sys

class Dinic:
    def __init__(self, n):

```

```

self.g = [[] for _ in range(n)]

def add_edge(self, u, v, cap):
    self.g[u].append([v, cap, len(self.g[v])])
    self.g[v].append([u, 0, len(self.g[u]) - 1])

def flow(self, s, t):
    answer = 0
    while True:
        level = [-1] * len(self.g)
        level[s] = 0
        q = collections.deque([s])
        while q:
            u = q.popleft()
            for v, cap, _ in self.g[u]:
                if cap and level[v] < 0:
                    level[v] = level[u] + 1
                    q.append(v)
        if level[t] < 0:
            return answer
        it = [0] * len(self.g)

    def dfs(u, pushed):
        if u == t:
            return pushed
        while it[u] < len(self.g[u]):
            e = self.g[u][it[u]]
            v, cap, rev = e
            if cap and level[v] == level[u] + 1:
                take = dfs(v, min(pushed, cap))
                if take:
                    e[1] -= take
                    self.g[v][rev][1] += take
                    return take
            it[u] += 1
        return 0

    while pushed := dfs(s, 10**30):
        answer += pushed

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    rows, cols, m = data[:3]
    at = 3
    demands = data[at : at + rows]
    at += rows
    bounds = []
    for _ in range(cols):

```

```

        bounds.append(tuple(data[at : at + 2]))
        at += 2
n = rows + cols + 2
source, sink, ss, tt = rows + cols, rows + cols + 1, n, n + 1
dinic = Dinic(n + 2)
balance = [0] * n

def bounded(u, v, low, high):
    dinic.add_edge(u, v, high - low)
    balance[u] -= low
    balance[v] += low

total = sum(demands)
for r, demand in enumerate(demands):
    bounded(source, r, demand, demand)
for c, (low, high) in enumerate(bounds):
    bounded(rows + c, sink, low, high)
for _ in range(m):
    r, c = data[at : at + 2]
    at += 2
    bounded(r - 1, rows + c - 1, 0, 1)
bounded(sink, source, total, total)
need = 0
for v, value in enumerate(balance):
    if value > 0:
        dinic.add_edge(ss, v, value)
        need += value
    elif value < 0:
        dinic.add_edge(v, tt, -value)
print("YES" if dinic.flow(ss, tt) == need else "NO")

if __name__ == "__main__":
    main()

```

3 B. Bounded Exact Shipment

Add a fixed return edge $t \rightarrow s$ carrying exactly F . The requested shipment is now a circulation. For every bounded edge, pre-send its lower bound, record the resulting vertex imbalances and its mandatory cost, and retain capacity high-low .

Connect a super-source to positive-imbalance vertices and negative-imbalance vertices to a super-sink. A minimum-cost flow saturating all super-source edges repairs conservation at minimum additional cost. If it cannot send the total imbalance, the shipment is impossible.

The lower-bound transformation is bijective: subtracting lower flows maps every feasible exact shipment to one repairing flow, and adding them back reverses the map. Costs differ only by the recorded mandatory constant, so minimum cost is preserved. With at most the total imbalance augmentations, the SPFA-based reference runs in $O(FVE)$ for these small flow bounds.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = (1LL << 62);

struct MinCostFlow {
    struct Edge { int to, rev, cap; ll cost; };
    vector<vector<Edge>> graph;

    explicit MinCostFlow(int n) : graph(n) {}

    void add_edge(int u, int v, int cap, ll cost) {
        int a = graph[u].size();
        int b = graph[v].size();
        graph[u].push_back({v, b, cap, cost});
        graph[v].push_back({u, a, 0, -cost});
    }

    pair<int, ll> flow(int source, int sink, int required) {
        int sent = 0;
        ll cost = 0;
        int n = graph.size();
        vector<ll> distance(n);
        vector<int> parent_v(n), parent_e(n);

        while (sent < required) {
            fill(distance.begin(), distance.end(), INF);
            vector<char> queued(n, false);
            queue<int> queue;
            distance[source] = 0;
            queue.push(source);
```

```

        queued[source] = true;
        while (!queue.empty()) {
            int u = queue.front();
            queue.pop();
            queued[u] = false;
            for (int i = 0; i < (int)graph[u].size(); ++i) {
                const Edge& edge = graph[u][i];
                if (!edge.cap) continue;
                ll candidate = distance[u] + edge.cost;
                if (candidate >= distance[edge.to]) continue;
                distance[edge.to] = candidate;
                parent_v[edge.to] = u;
                parent_e[edge.to] = i;
                if (!queued[edge.to]) {
                    queued[edge.to] = true;
                    queue.push(edge.to);
                }
            }
        }
        if (distance[sink] == INF) break;
        int add = required - sent;
        for (int v = sink; v != source; v = parent_v[v])
            add = min(add,
                graph[parent_v[v]][parent_e[v]].cap);
        for (int v = sink; v != source; v = parent_v[v]) {
            Edge& edge = graph[parent_v[v]][parent_e[v]];
            cost += ll(add) * edge.cost;
            edge.cap -= add;
            graph[v][edge.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, required_flow;
    cin >> n >> m >> required_flow;
    int source_vertex, sink_vertex;
    cin >> source_vertex >> sink_vertex;
    --source_vertex;
    --sink_vertex;

    int super_source = n;
    int super_sink = n + 1;
    MinCostFlow network(n + 2);
    vector<int> balance(n, 0);

```

```

ll mandatory_cost = 0;

auto add_bounded = [&](int u, int v, int low,
                      int high, ll cost) {
    balance[u] -= low;
    balance[v] += low;
    mandatory_cost += ll(low) * cost;
    network.add_edge(u, v, high - low, cost);
};

for (int i = 0; i < m; ++i) {
    int u, v, low, high;
    ll cost;
    cin >> u >> v >> low >> high >> cost;
    add_bounded(u - 1, v - 1, low, high, cost);
}
add_bounded(sink_vertex, source_vertex,
            required_flow, required_flow, 0);

int required = 0;
for (int vertex = 0; vertex < n; ++vertex) {
    if (balance[vertex] > 0) {
        network.add_edge(super_source, vertex,
                        balance[vertex], 0);
        required += balance[vertex];
    } else if (balance[vertex] < 0) {
        network.add_edge(vertex, super_sink,
                        -balance[vertex], 0);
    }
}

auto [sent, extra_cost] =
    network.flow(super_source, super_sink, required);
if (sent != required)
    cout << "IMPOSSIBLE\n";
else
    cout << mandatory_cost + extra_cost << '\n';
}

```

```

import collections
import sys

class MinCostFlow:
    def __init__(self, n):
        self.graph = [[] for _ in range(n)]

    def add_edge(self, u, v, capacity, cost):
        self.graph[u].append([v, len(self.graph[v]), capacity, cost])
        self.graph[v].append([u, len(self.graph[u]) - 1, 0, -cost])

```

```

def flow(self, source, sink, required):
    sent = total_cost = 0
    n = len(self.graph)
    while sent < required:
        distance = [None] * n
        parent = [None] * n
        queued = [False] * n
        queue = collections.deque([source])
        distance[source] = 0
        queued[source] = True
        while queue:
            u = queue.popleft()
            queued[u] = False
            for index, edge in enumerate(self.graph[u]):
                v, _, capacity, cost = edge
                if not capacity:
                    continue
                candidate = distance[u] + cost
                if distance[v] is not None and candidate >= distance[v]:
                    continue
                distance[v] = candidate
                parent[v] = (u, index)
                if not queued[v]:
                    queued[v] = True
                    queue.append(v)
            if distance[sink] is None:
                break
        add = required - sent
        vertex = sink
        while vertex != source:
            u, index = parent[vertex]
            add = min(add, self.graph[u][index][2])
            vertex = u
        vertex = sink
        while vertex != source:
            u, index = parent[vertex]
            edge = self.graph[u][index]
            total_cost += add * edge[3]
            edge[2] -= add
            self.graph[vertex][edge[1]][2] += add
            vertex = u
        sent += add
    return sent, total_cost

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m, exact = data[:3]
    source, sink = data[3] - 1, data[4] - 1
    at = 5
    network = MinCostFlow(n + 2)

```

```

balance = [0] * n
mandatory_cost = 0

def add_bounded(u, v, low, high, cost):
    nonlocal mandatory_cost
    balance[u] -= low
    balance[v] += low
    mandatory_cost += low * cost
    network.add_edge(u, v, high - low, cost)

for _ in range(m):
    u, v, low, high, cost = data[at:at + 5]
    at += 5
    add_bounded(u - 1, v - 1, low, high, cost)
add_bounded(sink, source, exact, exact, 0)

super_source, super_sink = n, n + 1
required = 0
for vertex, value in enumerate(balance):
    if value > 0:
        network.add_edge(super_source, vertex, value, 0)
        required += value
    elif value < 0:
        network.add_edge(vertex, super_sink, -value, 0)

sent, extra_cost = network.flow(super_source, super_sink, required)
if sent != required:
    print("IMPOSSIBLE")
else:
    print(mandatory_cost + extra_cost)

if __name__ == '__main__':
    main()

```

4 C. Selective Quota Profit

Send only K units from the source through worker vertices. Worker capacity one means a worker is selected at most once; eligibility edges choose a project. Project-to-sink capacity enforces upper quotas. Give the first $\text{low}[j]$ slots a reward $-\text{BIG}$, then verify that every rewarded slot was used.

4.1 How to find it

The flow value performs the selection: unused workers simply carry no flow. Eligibility costs are negative profits. BIG exceeds the complete possible profit difference, making lower-quota satisfaction lexicographically prior to profit maximization.

4.2 Correctness

Every integral value- K flow selects exactly K distinct workers and one eligible project for each. Conversely every valid selective assignment gives such a flow. Upper capacities and verified mandatory slots enforce both quota bounds. After removing the constant reward, minimizing cost is exactly maximizing assignment profit.

4.3 Complexity and tests

There are $O(n+p)$ vertices and $O(n+m+p)$ edges and K augmentations. Test $K=0$, workers left unused, tight quotas, and infeasible lower quotas.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll INF = (1LL << 62);

struct MCF {
    struct Edge { int to, rev, cap; ll cost; };
    vector<vector<Edge>> graph;
    explicit MCF(int n) : graph(n) {}

    pair<int, int> add_edge(int u, int v, int cap, ll cost) {
        int index = graph[u].size();
```

```

graph[u].push_back({v, (int)graph[v].size(), cap, cost});
graph[v].push_back({u, index, 0, -cost});
return {u, index};
}

pair<int, ll> flow(int source, int sink, int limit) {
    int n = graph.size();
    vector<ll> potential(n, INF), distance(n);
    potential[source] = 0;
    for (int repeat = 0; repeat < n; ++repeat) {
        bool changed = false;
        for (int u = 0; u < n; ++u) {
            if (potential[u] == INF) continue;
            for (const Edge& edge : graph[u]) {
                if (edge.cap
                    && potential[edge.to] > potential[u] + edge.cost) {
                    potential[edge.to] = potential[u] + edge.cost;
                    changed = true;
                }
            }
        }
        if (!changed) break;
    }
    for (ll& value : potential)
        if (value == INF) value = 0;

    int sent = 0;
    ll total_cost = 0;
    vector<int> parent_v(n), parent_e(n);
    while (sent < limit) {
        fill(distance.begin(), distance.end(), INF);
        distance[source] = 0;
        priority_queue<pair<ll, int>,
            vector<pair<ll, int>>, greater<pair<ll, int>>> queue;
        queue.push({0, source});
        while (!queue.empty()) {
            auto [current, u] = queue.top();
            queue.pop();
            if (current != distance[u]) continue;
            for (int i = 0; i < (int)graph[u].size(); ++i) {
                const Edge& edge = graph[u][i];
                if (!edge.cap) continue;
                ll next = current + edge.cost
                    + potential[u] - potential[edge.to];
                if (next >= distance[edge.to]) continue;
                distance[edge.to] = next;
                parent_v[edge.to] = u;
                parent_e[edge.to] = i;
                queue.push({next, edge.to});
            }
        }
    }
}

```

```

        if (distance[sink] == INF) break;
        for (int v = 0; v < n; ++v)
            if (distance[v] < INF) potential[v] += distance[v];
        int add = limit - sent;
        for (int v = sink; v != source; v = parent_v[v])
            add = min(add,
                graph[parent_v[v]][parent_e[v]].cap);
        for (int v = sink; v != source; v = parent_v[v]) {
            Edge& edge = graph[parent_v[v]][parent_e[v]];
            total_cost += ll(add) * edge.cost;
            edge.cap -= add;
            graph[v][edge.rev].cap += add;
        }
        sent += add;
    }
    return {sent, total_cost};
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, projects, m, chosen;
    cin >> n >> projects >> m >> chosen;
    vector<int> low(projects), high(projects);
    int mandatory_count = 0;
    for (int j = 0; j < projects; ++j) {
        cin >> low[j] >> high[j];
        mandatory_count += low[j];
    }

    struct Choice { int worker, project, profit; };
    vector<Choice> choices(m);
    int maximum_profit = 0;
    for (Choice& choice : choices) {
        cin >> choice.worker >> choice.project >> choice.profit;
        --choice.worker;
        --choice.project;
        maximum_profit = max(maximum_profit, choice.profit);
    }

    int source = n + projects;
    int sink = source + 1;
    MCF network(sink + 1);
    for (int worker = 0; worker < n; ++worker)
        network.add_edge(source, worker, 1, 0);
    for (const Choice& choice : choices)
        network.add_edge(choice.worker, n + choice.project,
            1, -choice.profit);

    ll big = ll(chosen + 1) * (maximum_profit + 1LL);

```

```

vector<pair<int, int>> mandatory;
for (int project = 0; project < projects; ++project) {
    mandatory.push_back(network.add_edge(
        n + project, sink, low[project], -big));
    network.add_edge(n + project, sink,
        high[project] - low[project], 0);
}

auto [sent, cost] = network.flow(source, sink, chosen);
bool possible = sent == chosen;
for (auto [u, index] : mandatory)
    possible &= network.graph[u][index].cap == 0;
if (!possible)
    cout << "IMPOSSIBLE\n";
else
    cout << -cost - big * mandatory_count << '\n';
}

```

4.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):
        self.graph = [[] for _ in range(n)]

    def add_edge(self, u, v, capacity, cost):
        reference = (u, len(self.graph[u]))
        self.graph[u].append([v, capacity, cost, len(self.graph[v])])
        self.graph[v].append([u, 0, -cost, len(self.graph[u]) - 1])
        return reference

    def flow(self, source, sink, limit):
        n = len(self.graph)
        infinity = 10**40
        potential = [infinity] * n
        potential[source] = 0
        for _ in range(n):
            changed = False
            for u in range(n):
                if potential[u] == infinity:
                    continue
                for v, capacity, cost, _ in self.graph[u]:
                    if capacity and potential[v] > potential[u] + cost:
                        potential[v] = potential[u] + cost
                        changed = True

```

```

        if not changed:
            break
    potential = [0 if value == infinity else value for value in potential]

    sent = total_cost = 0
    while sent < limit:
        distance = [infinity] * n
        parent = [None] * n
        distance[source] = 0
        queue = [(0, source)]
        while queue:
            current, u = heapq.heappop(queue)
            if current != distance[u]:
                continue
            for index, edge in enumerate(self.graph[u]):
                v, capacity, cost, _ = edge
                if not capacity:
                    continue
                candidate = current + cost + potential[u] - potential[v]
                if candidate < distance[v]:
                    distance[v] = candidate
                    parent[v] = (u, index)
                    heapq.heappush(queue, (candidate, v))
            if distance[sink] == infinity:
                break
        for v in range(n):
            if distance[v] < infinity:
                potential[v] += distance[v]
        add = limit - sent
        vertex = sink
        while vertex != source:
            u, index = parent[vertex]
            add = min(add, self.graph[u][index][1])
            vertex = u
        vertex = sink
        while vertex != source:
            u, index = parent[vertex]
            edge = self.graph[u][index]
            total_cost += add * edge[2]
            edge[1] -= add
            self.graph[vertex][edge[3]][1] += add
            vertex = u
        sent += add
    return sent, total_cost

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, projects, m, chosen = data[:4]
    at = 4
    bounds = []

```

```

for _ in range(projects):
    bounds.append(tuple(data[at:at + 2]))
    at += 2
choices = []
maximum_profit = 0
for _ in range(m):
    worker, project, profit = data[at:at + 3]
    at += 3
    choices.append((worker - 1, project - 1, profit))
    maximum_profit = max(maximum_profit, profit)

source = n + projects
sink = source + 1
network = MCF(sink + 1)
for worker in range(n):
    network.add_edge(source, worker, 1, 0)
for worker, project, profit in choices:
    network.add_edge(worker, n + project, 1, -profit)

big = (chosen + 1) * (maximum_profit + 1)
mandatory = []
for project, (low, high) in enumerate(bounds):
    mandatory.append(
        network.add_edge(n + project, sink, low, -big)
    )
    network.add_edge(n + project, sink, high - low, 0)

sent, cost = network.flow(source, sink, chosen)
possible = sent == chosen and all(
    network.graph[u][index][1] == 0 for u, index in mandatory
)
if possible:
    print(-cost - big * sum(low for low, _ in bounds))
else:
    print("IMPOSSIBLE")

if __name__ == '__main__':
    main()

```

5 D. Bounded Convex Allocation

First assign every machine its lower bound. For machine i , the cost of its j -th additional unit is

```
a[i] * (2 * (low[i] + j) - 1) + b[i].
```

These marginal costs form an increasing arithmetic sequence. The remaining task is to select the globally smallest K -sum(low) marginals, respecting the finite length of each sequence.

5.1 How to find it

For a threshold T , count in constant time how many terms of one arithmetic sequence are at most T . Summing this over machines gives a monotone predicate. Binary-search the first threshold covering the required count, sum every term strictly below it with the arithmetic-series formula, then take the remaining tied terms at the threshold.

5.2 Correctness

Discrete convexity makes each machine's marginals nondecreasing, so any optimal allocation using x extra units from it uses its first x marginals. Thus allocations correspond exactly to prefix selections from these sequences. Selecting all values below the smallest covering threshold and any required tied values is precisely the globally cheapest such selection.

5.3 Complexity and tests

Threshold counting costs $O(n)$ and needs $O(\log C)$ iterations, where C is the marginal-value range. Memory is $O(n)$. Test infeasible totals, negative marginals, nonzero lower bounds, huge capacities, and many equal marginals.

5.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using i128 = __int128_t;
using int64 = long long;

struct Machine {
    i128 a, start;
```

```

    int64 capacity;
};

void print_i128(i128 value) {
    if (value == 0) {
        cout << "0\n";
        return;
    }
    if (value < 0) {
        cout << '-';
        value = -value;
    }
    string digits;
    while (value) {
        digits.push_back(char('0' + value % 10));
        value /= 10;
    }
    reverse(digits.begin(), digits.end());
    cout << digits << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    int64 wanted;
    cin >> n >> wanted;
    vector<Machine> machines;
    i128 base_cost = 0;
    i128 minimum_total = 0;
    i128 maximum_total = 0;
    i128 search_low = 0, search_high = 0;
    bool has_extra = false;

    for (int i = 0; i < n; ++i) {
        int64 a_input, b_input, low, high;
        cin >> a_input >> b_input >> low >> high;
        i128 a = a_input;
        i128 b = b_input;
        base_cost += a * low * low + b * low;
        minimum_total += low;
        maximum_total += high;
        int64 capacity = high - low;
        if (!capacity) continue;
        i128 start = a * (2 * i128(low) + 1) + b;
        i128 last = start + 2 * a * (capacity - 1);
        machines.push_back({a, start, capacity});
        if (!has_extra) {
            search_low = start;
            search_high = last;
        }
    }
}

```

```

        has_extra = true;
    } else {
        search_low = min(search_low, start);
        search_high = max(search_high, last);
    }
}

if (wanted < minimum_total || wanted > maximum_total) {
    cout << "IMPOSSIBLE\n";
    return 0;
}

int64 need = wanted - int64(minimum_total);
if (!need) {
    print_i128(base_cost);
    return 0;
}

auto count_at_most = [&](i128 threshold) {
    int64 count = 0;
    for (const Machine& machine : machines) {
        if (threshold < machine.start) continue;
        i128 raw = (threshold - machine.start)
            / (2 * machine.a) + 1;
        int64 take = raw > machine.capacity
            ? machine.capacity : int64(raw);
        if (count >= need - take) return need;
        count += take;
    }
    return count;
};

i128 left = search_low;
i128 right = search_high;
while (left < right) {
    i128 middle = left + (right - left) / 2;
    if (count_at_most(middle) >= need)
        right = middle;
    else
        left = middle + 1;
}
i128 threshold = left;

int64 selected = 0;
i128 answer = base_cost;
for (const Machine& machine : machines) {
    if (threshold <= machine.start) continue;
    i128 raw = (threshold - 1 - machine.start)
        / (2 * machine.a) + 1;
    int64 take = raw > machine.capacity
        ? machine.capacity : int64(raw);
    selected += take;
}

```

```

        answer += i128(take) * machine.start
                + machine.a * take * (take - 1);
    }
    answer += i128(need - selected) * threshold;
    print_i128(answer);
}

```

5.5 Python

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, wanted = data[:2]
    at = 2
    machines = []
    base_cost = 0
    minimum_total = maximum_total = 0
    for _ in range(n):
        a, b, low, high = data[at:at + 4]
        at += 4
        base_cost += a * low * low + b * low
        minimum_total += low
        maximum_total += high
        capacity = high - low
        if capacity:
            start = a * (2 * low + 1) + b
            last = start + 2 * a * (capacity - 1)
            machines.append((a, start, capacity, last))

    if not minimum_total <= wanted <= maximum_total:
        print("IMPOSSIBLE")
        return
    need = wanted - minimum_total
    if need == 0:
        print(base_cost)
        return

    def count_at_most(threshold):
        count = 0
        for a, start, capacity, _ in machines:
            if threshold >= start:
                count += min(capacity, (threshold - start) // (2 * a) + 1)
            if count >= need:
                return need
        return count

```

```

left = min(start for _, start, _, _ in machines)
right = max(last for _, _, _, last in machines)
while left < right:
    middle = (left + right) // 2
    if count_at_most(middle) >= need:
        right = middle
    else:
        left = middle + 1
threshold = left

answer = base_cost
selected = 0
for a, start, capacity, _ in machines:
    if threshold <= start:
        continue
    take = min(capacity, (threshold - 1 - start) // (2 * a) + 1)
    selected += take
    answer += take * start + a * take * (take - 1)
answer += (need - selected) * threshold
print(answer)

if __name__ == '__main__':
    main()

```

6 E. Exact Red Forest

Price every red edge by `lambda`. Kruskal, stopped after `n-components` accepted edges, solves the penalized fixed-size forest problem. The red count is nonincreasing in `lambda`; opposite tie rules give the extreme red counts among equally optimal forests.

6.1 How to find it

The difficult condition is the exact red count. A forest with the requested component count has fixed size, so truncated Kruskal is still the optimizer after adding a charge per red edge.

6.2 Correctness

Fixed-size forests are bases of a truncated graphic matroid. Its exchange property gives the same tie interval as for spanning trees: all red counts between the two optimal tie extremes are attainable. Binary search finds the boundary price whose interval contains `k`; subtracting `lambda*k` recovers the minimum original weight.

6.3 Complexity and tests

Each price evaluation sorts `m` edges, giving $O(m \log m \log W)$. Test zero selected edges, disconnected graphs with enough forest rank, impossible color counts, negative weights, and many modified-cost ties.

6.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct DSU {
    vector<int> parent, size;
    explicit DSU(int n) : parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int value) {
        while (value != parent[value]) value = parent[value];
        return value;
    }
}
```

```

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        return true;
    }
};

struct Edge { int u, v, red; ll weight; };

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, components, wanted;
    cin >> n >> m >> components >> wanted;
    int edge_count = n - components;
    vector<Edge> edges(m);
    for (Edge& edge : edges) {
        char color;
        cin >> edge.u >> edge.v >> edge.weight >> color;
        --edge.u;
        --edge.v;
        edge.red = color == 'R';
    }
    if (edge_count == 0) {
        cout << (wanted == 0 ? "0\n" : "IMPOSSIBLE\n");
        return 0;
    }

    auto run = [&](ll penalty, bool prefer_red) {
        vector<int> order(m);
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) {
            ll first = edges[i].weight + penalty * edges[i].red;
            ll second = edges[j].weight + penalty * edges[j].red;
            if (first != second) return first < second;
            return prefer_red ? edges[i].red > edges[j].red
                               : edges[i].red < edges[j].red;
        });
        DSU dsu(n);
        int used = 0;
        int red = 0;
        ll value = 0;
        for (int index : order) {
            const Edge& edge = edges[index];
            if (!dsu.unite(edge.u, edge.v)) continue;
            ++used;

```

```

        red += edge.red;
        value += edge.weight + penalty * edge.red;
        if (used == edge_count) break;
    }
    return tuple<int, int, ll>{used, red, value};
};

const ll BOUND = 400'000'000'000'000LL;
auto [minimum_used, minimum_red, ignored1] = run(BOUND, false);
auto [maximum_used, maximum_red, ignored2] = run(-BOUND, true);
if (minimum_used != edge_count || maximum_used != edge_count
    || wanted < minimum_red || wanted > maximum_red) {
    cout << "IMPOSSIBLE\n";
    return 0;
}

ll low = -BOUND;
ll high = BOUND;
while (low < high) {
    ll middle = low + (high - low + 1) / 2;
    auto [used, red, value] = run(middle, true);
    if (red >= wanted)
        low = middle;
    else
        high = middle - 1;
}
auto [used, red, modified] = run(low, true);
cout << modified - low * wanted << '\n';
}

```

6.5 Python

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    iterator = iter(data)
    n = int(next(iterator))
    m = int(next(iterator))
    components = int(next(iterator))
    wanted = int(next(iterator))
    edges = []
    for _ in range(m):
        u = int(next(iterator)) - 1
        v = int(next(iterator)) - 1
        weight = int(next(iterator))
        red = next(iterator) == b'R'
        edges.append((u, v, weight, red))

```

```

edge_count = n - components
if edge_count == 0:
    print(0 if wanted == 0 else "IMPOSSIBLE")
    return

def run(penalty, prefer_red):
    order = sorted(
        edges,
        key=lambda edge: (
            edge[2] + penalty * edge[3],
            -edge[3] if prefer_red else edge[3],
        ),
    )
    parent = list(range(n))
    size = [1] * n

    def find(value):
        while value != parent[value]:
            value = parent[value]
        return value

    used = red_count = value = 0
    for u, v, weight, red in order:
        u, v = find(u), find(v)
        if u == v:
            continue
        if size[u] < size[v]:
            u, v = v, u
        parent[v] = u
        size[u] += size[v]
        used += 1
        red_count += red
        value += weight + penalty * red
        if used == edge_count:
            break
    return used, red_count, value

bound = 400_000_000_000_000
minimum_used, minimum_red, _ = run(bound, False)
maximum_used, maximum_red, _ = run(-bound, True)
if (minimum_used != edge_count or maximum_used != edge_count
    or not minimum_red <= wanted <= maximum_red):
    print("IMPOSSIBLE")
    return

low, high = -bound, bound
while low < high:
    middle = (low + high + 1) // 2
    if run(middle, True)[1] >= wanted:
        low = middle
    else:

```

```
        high = middle - 1
    _, _, modified = run(low, True)
    print(modified - low * wanted)

if __name__ == '__main__':
    main()
```

7 F. Congested Team Assignment

Worker-to-team edges contribute negative profit. Replace team cost $a \cdot x^2$ by x unit slots with marginal costs $a(2k-1)$. Reward the mandatory lower-quota slots by $-BIG$ and verify them afterward. A single exact min-cost flow now optimizes assignment and congestion together.

7.1 How to find it

This is assignment flow except the price of a team depends on its final load. Convexity makes the dependence separable: the k -th member costs the marginal $a[j](2k-1)$. Replace one nonlinear team edge by ordered unit slot edges, and use the lower-quota reward from the quota problem on its first slots.

7.2 Correctness

An integral value- n flow chooses one eligible team per specialist. Since a team's marginal slot costs are nondecreasing, any cheapest flow using x slots uses precisely the first x ; their sum is $a[j]x^2$. The $-BIG$ reward, chosen larger than the full real-objective range, makes saturation of every lower-quota slot lexicographically prior to profit and congestion. After the explicit saturation check, feasible flows and valid assignments correspond bijectively and have equal real objective, so the returned maximum is exact.

7.3 Complexity and tests

There are $O(n+p)$ vertices and $O(m+n+\text{sum high})$ edges, with at most n augmentations. Test zero congestion, tight project sizes, a quota forcing an unprofitable assignment, equal marginal slots, and infeasible eligibility.

7.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
const int64 INF = (1LL << 62);

struct MCF {
    struct Edge {
        int to, rev, cap, initial;
        int64 cost;
```

```

};
vector<vector<Edge>> g;
MCF(int n) : g(n) {}
pair<int, int> add_edge(int u, int v, int cap, int64 cost) {
    int id = g[u].size();
    g[u].push_back({v, (int)g[v].size(), cap, cap, cost});
    g[v].push_back({u, id, 0, 0, -cost});
    return {u, id};
}
pair<int, int64> flow(int s, int t, int limit) {
    int n = g.size(), sent = 0;
    int64 cost = 0;
    vector<int64> pot(n, INF), dist(n);
    pot[s] = 0;
    for (int rep = 0; rep < n; ++rep) {
        bool changed = false;
        for (int u = 0; u < n; ++u)
            if (pot[u] < INF)
                for (auto &e : g[u])
                    if (e.cap && pot[e.to] > pot[u] + e.cost)
                        pot[e.to] = pot[u] + e.cost, changed = true;
        if (!changed)
            break;
    }
    for (auto &x : pot)
        if (x == INF)
            x = 0;
    vector<int> pv(n), pe(n);
    while (sent < limit) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<int64, int>, vector<pair<int64, int>>,
            greater<pair<int64, int>>>
            pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (d != dist[u])
                continue;
            for (int i = 0; i < (int)g[u].size(); ++i) {
                auto &e = g[u][i];
                int64 nd = d + e.cost + pot[u] - pot[e.to];
                if (e.cap && nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i, pq.push({nd, e.to});
            }
        }
    }
    if (dist[t] == INF)
        break;
    for (int v = 0; v < n; ++v)
        if (dist[v] < INF)

```

```

        pot[v] += dist[v];
        int add = limit - sent;
        for (int v = t; v != s; v = pv[v])
            add = min(add, g[pv[v]][pe[v]].cap);
        for (int v = t; v != s; v = pv[v]) {
            Edge &e = g[pv[v]][pe[v]];
            cost += (int64)add * e.cost;
            e.cap -= add;
            g[v][e.rev].cap += add;
        }
        sent += add;
    }
    return {sent, cost};
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, p, m;
    if (!(cin >> n >> p >> m))
        return 0;
    vector<int> low(p), high(p), a(p);
    int required = 0, max_a = 0;
    for (int j = 0; j < p; ++j)
        cin >> low[j] >> high[j] >> a[j], required += low[j],
        max_a = max(max_a, a[j]);
    struct Choice {
        int w, j, profit;
    };
    vector<Choice> choices(m);
    int max_profit = 0;
    for (auto &[w, j, profit] : choices)
        cin >> w >> j >> profit, --w, --j, max_profit = max(max_profit, profit);
    int s = n + p, t = s + 1;
    MCF mcf(t + 1);
    for (int w = 0; w < n; ++w)
        mcf.add_edge(s, w, 1, 0);
    for (auto [w, j, profit] : choices)
        mcf.add_edge(w, n + j, 1, -profit);
    int64 big = (int64)(n + 1) * (max_profit + 2LL * max_a * n + 1);
    vector<pair<int, int>> mandatory;
    for (int j = 0; j < p; ++j)
        for (int k = 1; k <= high[j]; ++k) {
            int64 cost = (int64)a[j] * (2LL * k - 1);
            auto ref = mcf.add_edge(n + j, t, 1, cost - (k <= low[j] ? big : 0));
            if (k <= low[j])
                mandatory.push_back(ref);
        }
    auto [sent, cost] = mcf.flow(s, t, n);
    bool ok = sent == n;

```

```

for (auto [u, id] : mandatory)
    ok &= mcf.g[u][id].cap == 0;
if (!ok)
    cout << "IMPOSSIBLE\n";
else
    cout << -cost - big * required << '\n';
}

```

7.5 Python

```

import heapq
import sys

class MCF:
    def __init__(self, n):
        self.g = [[] for _ in range(n)]

    def add_edge(self, u, v, cap, cost):
        ref = (u, len(self.g[u]))
        self.g[u].append([v, cap, cost, len(self.g[v]), cap])
        self.g[v].append([u, 0, -cost, len(self.g[u]) - 1, 0])
        return ref

    def flow(self, s, t, limit):
        n = len(self.g)
        inf = 10**40
        potential = [inf] * n
        potential[s] = 0
        for _ in range(n):
            changed = False
            for u in range(n):
                if potential[u] == inf:
                    continue
                for v, cap, cost, _, _ in self.g[u]:
                    if cap and potential[v] > potential[u] + cost:
                        potential[v] = potential[u] + cost
                        changed = True
            if not changed:
                break
        potential = [0 if x == inf else x for x in potential]
        sent = total = 0
        while sent < limit:
            dist = [inf] * n
            prev = [None] * n
            dist[s] = 0
            pq = [(0, s)]
            while pq:

```

```

        d, u = heapq.heappop(pq)
        if d != dist[u]:
            continue
        for i, (v, cap, cost, _, _) in enumerate(self.g[u]):
            nd = d + cost + potential[u] - potential[v]
            if cap and nd < dist[v]:
                dist[v] = nd
                prev[v] = (u, i)
                heapq.heappush(pq, (nd, v))
    if dist[t] == inf:
        break
    for v in range(n):
        if dist[v] < inf:
            potential[v] += dist[v]
    add = limit - sent
    v = t
    while v != s:
        u, i = prev[v]
        add = min(add, self.g[u][i][1])
        v = u
    v = t
    while v != s:
        u, i = prev[v]
        e = self.g[u][i]
        total += add * e[2]
        e[1] -= add
        self.g[v][e[3]][1] += add
        v = u
    sent += add
    return sent, total

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, p, m = data[:3]
    at = 3
    projects = []
    for _ in range(p):
        projects.append(tuple(data[at : at + 3]))
        at += 3
    choices = []
    max_profit = 0
    for _ in range(m):
        w, j, profit = data[at : at + 3]
        at += 3
        choices.append((w - 1, j - 1, profit))
        max_profit = max(max_profit, profit)
    s, t = n + p, n + p + 1
    mcf = MCF(t + 1)

```

```

for w in range(n):
    mcf.add_edge(s, w, 1, 0)
for w, j, profit in choices:
    mcf.add_edge(w, n + j, 1, -profit)
max_a = max(a for _, _, a in projects)
big = (n + 1) * (max_profit + 2 * max_a * n + 1)
mandatory = []
for j, (low, high, a) in enumerate(projects):
    for k in range(1, high + 1):
        ref = mcf.add_edge(
            n + j, t, 1, a * (2 * k - 1) - (big if k <= low else 0)
        )
        if k <= low:
            mandatory.append(ref)
sent, cost = mcf.flow(s, t, n)
ok = sent == n and all(mcf.g[u][i][1] == 0 for u, i in mandatory)
print(
    -cost - big * sum(low for low, _, _ in projects)
    if ok
    else "IMPOSSIBLE"
)

if __name__ == "__main__":
    main()

```